

# Credit-Aware Counterfactual Rollout Allocation for LLM Reasoning

Anonymous ACL submission

## Abstract

Reinforcement learning (RL) for large language model (LLM) reasoning commonly provides feedback only at the end of a reasoning trajectory, making it difficult to determine which intermediate decisions caused the final outcome. Tree-based approaches address this problem by sharing intermediate reasoning states across multiple trajectories, but when the number of rollouts is fixed, due to rollout generation being a computationally costly process, an important question remains: where should additional trajectories be generated to obtain the most useful data? We propose a two-stage adaptive rollout framework that separates exploration from exploitation. In the first stage,  $n - k$  trajectories, where  $n$  is the total number of rollouts that can be generated, are used to construct an exploratory reasoning tree using a reward-independent heuristic based on the continuation cross-entropy of observed future trajectories. In the second stage, the initial tree is analyzed using tree-based value estimates and local temporal-difference differences to identify consequential decisions. The remaining  $k$  rollouts are then allocated to the corresponding parent states to obtain targeted counterfactual continuations. The resulting tree is used to derive variable, node-level credit estimates for policy optimization. We hypothesize that credit-aware allocation of a fixed rollout budget can provide more informative counterfactual evidence than uniform allocation without increasing the total number of trajectories.

## 1 Introduction

Reinforcement learning (RL) has become an effective approach for improving the reasoning capabilities of large language models (LLMs). However, reasoning tasks often provide feedback only through a terminal reward, such as whether the final answer is correct. This creates a fundamental *credit assignment* problem: a long reasoning trajectory may contain many intermediate decisions, while

the final reward does not indicate which decisions were responsible for the outcome.

A natural way to obtain more localized credit is to generate multiple trajectories that share intermediate reasoning states. When trajectories diverge from the same state and receive different terminal rewards, their outcomes provide counterfactual evidence about the consequences of decisions at that state. Recent methods exploit this structure for credit assignment and exploration, including TEMPO (Tran et al., 2025), TreePO (Li et al., 2025), and TreeRL (Hou et al., 2025). Other works investigate adaptive rollout allocation (Zhang et al., 2026; Zou et al., 2026) and the use of counterfactual reasoning paths for reducing credit-assignment variance (Ding et al., 2026).

However, since generating trajectories might prove to be computationally expensive, a limit on the number of rollout budgets is often imposed. These methods motivate the following question:

*Given a fixed rollout budget, where should additional trajectories be generated to obtain the most useful counterfactual evidence for credit assignment?*

Generating all trajectories independently or allocating them uniformly may spend computation on states whose outcomes provide little additional information. On the other hand, using terminal rewards to guide the initial exploration stage may bias exploration toward already-observed outcomes.

We therefore propose a two-stage strategy that separates *exploration* from *exploitation*. The first stage uses  $n - k$  trajectories to construct an initial reasoning tree using a reward-independent continuation cross-entropy heuristic. The second stage uses the outcomes in this tree to identify consequential decisions and allocates the remaining  $k$  trajectories to their corresponding parent states. These additional trajectories provide targeted counterfac-

tual evidence while keeping the total rollout budget fixed.

## 2 Background

### 2.1 Tree-Based Credit Assignment

Consider a reasoning trajectory

$$\tau = (v_0, v_1, \dots, v_m), \quad (1)$$

where each  $v_i$  represents a reasoning segment and the trajectory receives a terminal reward  $R(\tau)$ .

Tree-based rollout methods represent multiple trajectories as a tree whose nodes correspond to shared reasoning segments. For a node  $v$ , let  $\mathcal{L}(v)$  denote its descendant leaf trajectories. A simple empirical value estimate is

$$V(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} R_\ell. \quad (2)$$

When multiple trajectories share  $v$ , this value captures the observed outcomes associated with that state. Our method uses this tree structure not only for credit assignment, but also to determine where additional rollout computation should be spent.

## 3 Related Work

**Tree-based credit assignment.** TEMPO (Tran et al., 2025) exploits shared tree structure to estimate intermediate values from descendant outcomes and obtain more localized credit signals. TreePO (Li et al., 2025) uses segment-level tree rollouts for efficient reasoning optimization, while TreeRL (Hou et al., 2025) performs on-policy tree search by branching at uncertain intermediate states. Our method similarly uses shared reasoning trees, but focuses on allocating a fixed rollout budget between initial exploration and targeted counterfactual expansion.

**Adaptive rollout allocation.** IGRPO (Zhang et al., 2026) and TRACE (Zou et al., 2026) investigate adaptive allocation of rollout computation using signals intended to identify informative reasoning states. Our approach also allocates rollouts adaptively, but uses the initial exploratory tree to subsequently identify states based on their observed outcome consequences.

**Counterfactual reasoning paths.** Recent work shows that alternative reasoning paths can provide useful process-level information and reduce the variance of credit-assignment signals (Ding et al., 2026). We address the complementary problem of deciding where a limited number of such counterfactual paths should be generated.

## 4 Proposed Method

### 4.1 Problem Formulation

For each problem  $x$ , let  $n$  be the total rollout budget. We divide it into two portions:

$$n = (n - k) + k. \quad (3)$$

The first  $n - k$  trajectories are used for *exploration*, while the remaining  $k$  trajectories are reserved for *credit-guided counterfactual expansion*.

Each tree node represents a reasoning segment rather than an individual token. Thus, a trajectory consisting of segments

$$v_1, v_2, \dots, v_m$$

forms a path in the tree, and trajectories sharing a prefix reuse the corresponding nodes.

### 4.2 Phase I: Exploration-Guided Tree Construction

The first phase constructs an initial tree  $T_0$  using  $n - k$  trajectories. Trajectories are generated sequentially so that later trajectories can branch from states already present in the tree.

#### 4.2.1 Initial Trajectory

We first generate a complete trajectory from the original problem  $x$ . Every reasoning segment becomes a node in  $T_0$ . The initial tree is therefore a single path from the root to a terminal leaf.

For each intermediate node  $v$ , we consider the portion of the trajectory following  $v$  as its observed continuation.

#### 4.2.2 Continuation Cross-Entropy

Let  $c_v$  denote the model context at node  $v$ , and let  $S_{\ell,v}$  denote the suffix of trajectory  $\ell$  following  $v$ . If this suffix contains  $L_{\ell,v}$  tokens, its average negative log-likelihood is

$$H_{\ell,v} = -\frac{1}{L_{\ell,v}} \sum_{t=1}^{L_{\ell,v}} \log p_\theta(x_t | c_v, x_{<t}). \quad (4)$$

The corresponding continuation perplexity is

$$PPL_{\ell,v} = \exp(H_{\ell,v}). \quad (5)$$

Our heuristic is inspired by the intuition behind continuation perplexity: the likelihood of the future continuation provides information about how predictable that continuation is from an intermediate state. We use the underlying cross-entropy rather than directly averaging perplexity values.

As more trajectories pass through a node, let  $\mathcal{L}(v)$  denote its observed descendant leaves. We compute

Our heuristic is inspired by the intuition behind continuation perplexity: the likelihood of the future continuation provides information about how predictable that continuation is from an intermediate state. We use the underlying cross-entropy rather than directly averaging perplexity values.

As more trajectories pass through a node, let  $\mathcal{L}(v)$  denote its observed descendant leaves. We compute

$$\mu_v = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} H_{\ell,v}, \quad (6)$$

and

$$\sigma_v^2 = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} (H_{\ell,v} - \mu_v)^2. \quad (7)$$

The mean captures the average surprise of observed continuations, while the standard deviation captures their variability in predictability. We use both quantities because a useful exploratory state may be characterized not only by consistently difficult-to-predict continuations, but also by substantial variation across observed continuations. We therefore define the reward-independent exploration score as

$$S_{\text{explore}}(v) = \mu_v + \lambda \sigma_v, \quad (8)$$

where  $\lambda$  controls the relative importance of continuation variability.

This score favors nodes whose observed continuations are both difficult to predict on average and heterogeneous in their predictability. We treat this combination as a proxy for exploratory potential rather than assuming that high continuation cross-entropy necessarily implies useful alternative outcomes. No terminal reward is used in this score, keeping Phase I independent of observed task success.

### 4.2.3 Sequential Branching

After the first trajectory, each subsequent trajectory is generated by branching from the existing tree. At each iteration, eligible reasoning nodes are scored using  $S_{\text{explore}}$ , and a branching node is selected according to the exploration policy.

The model then generates a complete continuation from the selected state. The new trajectory shares the existing prefix and creates a new branch from that state. After insertion, continuation statistics are recomputed for the affected node and its ancestors.

This process continues until exactly  $n - k$  trajectories have been incorporated into  $T_0$ .

### 4.3 Phase II: Credit-Guided Counterfactual Expansion

After the exploratory tree has been constructed, terminal rewards become available. Phase II uses these outcomes to determine where the remaining  $k$  rollouts should be spent.

#### 4.3.1 Tree-Based Value Estimation

For each node  $v$ , its empirical value is

$$V(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} R_{\ell}. \quad (9)$$

For binary mathematical reasoning rewards,  $V(v)$  corresponds to the empirical success rate of trajectories passing through  $v$ .

#### 4.3.2 Local Consequentiality

We measure the change in estimated value across a reasoning decision using

$$\delta(v) = V(v) - V(\text{parent}(v)). \quad (10)$$

The parent state represents the point at which the decision leading to  $v$  was made. Thus,  $\delta(v)$  provides a local signal for how the observed outcome value changes across that decision.

We initially use this signal directly for rollout allocation.

#### 4.3.3 Counterfactual Allocation

Let  $\mathcal{V}(T_0)$  denote the candidate non-root nodes. We convert their consequentiality scores into an allocation distribution:

$$p(v_i) = \frac{\exp(\delta(v_i)/\tau)}{\sum_{v_j \in \mathcal{V}(T_0)} \exp(\delta(v_j)/\tau)}, \quad (11)$$

where  $\tau$  is a temperature parameter.  
 For each of the remaining  $k$  rollouts, a node  $v$  is selected according to this distribution. We then return to its parent,

$$p(v) = \text{parent}(v), \quad (12)$$

because the parent is the decision point at which the original branch was chosen. An independent continuation is generated from this state and attached to the existing tree as an additional child.

Thus, if the initial tree contains

$$p(v) \rightarrow v \rightarrow \dots \rightarrow \ell_1,$$

the expanded tree may contain

$$p(v) \rightarrow \begin{cases} v \rightarrow \dots \rightarrow \ell_1, \\ v' \rightarrow \dots \rightarrow \ell_2. \end{cases}$$

The original trajectory is preserved, while the new trajectory provides counterfactual evidence from the same decision state. This procedure continues until all  $k$  reserved rollouts are consumed.

#### 4.4 Final Credit Assignment and Policy Optimization

After counterfactual expansion, the tree values are recomputed to incorporate the additional trajectories. The complete tree is then used to obtain variable, node-level credit estimates.

Rather than assigning a uniform trajectory-level advantage to every reasoning decision, we derive credit signals from the tree structure and local value differences. These signals are incorporated into the advantage estimates used for policy optimization.

Thus, the final training procedure is a policy optimization method with *tree-derived, variable-credit advantage estimates*. The key distinction is that the credit assigned to an intermediate reasoning state depends on its observed position and consequences within the completed tree.

#### 4.5 Algorithm

Algorithm 1 summarizes the complete rollout allocation procedure.

The procedure uses exactly  $n - k$  exploratory trajectories and  $k$  targeted trajectories, maintaining the fixed total rollout budget  $n$ .

## 5 Dataset and Experimental Setup

### 5.1 Datasets

We will initially evaluate the method on verifiable mathematical reasoning tasks. Mathematical reasoning provides deterministic terminal rewards based on final-answer correctness, allowing controlled evaluation of credit assignment and rollout allocation.

MATH will be used for training, with MATH500 as a held-out evaluation benchmark. Competition-style datasets such as AIME and AMC will be used to evaluate generalization to more challenging mathematical problems.

### 5.2 Model and Training

We will initially use a relatively small open-weight reasoning-capable model to make repeated rollout experiments computationally feasible. We will consider Qwen-family models, consistent with recent tree-based reasoning work.

All methods will be evaluated under matched rollout budgets. In our method,  $n - k$  trajectories are allocated to exploration and  $k$  to targeted expansion. The policy optimization stage uses the resulting tree-derived, variable-credit node-level advantages.

## 6 Evaluation Plan

We will evaluate the method along three dimensions: reasoning performance, counterfactual allocation quality, and rollout efficiency.

**Reasoning performance.** We will measure final-answer accuracy on held-out mathematical reasoning benchmarks and compare against standard policy optimization and relevant tree-based approaches such as TEMPO, TreePO, and, where feasible, TreeRL under matched rollout budgets.

**Counterfactual allocation quality.** We will evaluate whether states selected for additional expansion provide more informative outcome variation than states selected by alternative strategies. On a smaller subset, we will generate substantially more rollouts from candidate states to approximate their possible outcome distributions. This provides an approximate oracle for evaluating whether our selected states exhibit greater downstream outcome variation.

**Exploration ablations.** We will compare Phase-I allocation using:

- random branching;
- mean continuation cross-entropy;
- variance of continuation cross-entropy;
- the proposed mean-plus-variance heuristic;
- other uncertainty-based alternatives where feasible.

**Counterfactual allocation ablations.** We will compare credit-guided allocation of the  $k$  additional rollouts against random or uniform allocation.

**Budget efficiency.** We will evaluate several total rollout budgets  $n$  and reserved-budget ratios  $k/n$ , measuring whether the proposed method provides better reasoning performance or credit localization at the same total computational budget.

## 7 Expected Contributions

The project aims to make three contributions.

First, we formulate adaptive rollout allocation as a *credit-aware exploration problem*, treating additional trajectories as a limited resource for obtaining counterfactual evidence.

Second, we propose a two-stage framework that separates reward-independent exploration from credit-aware exploitation. The first stage constructs an exploratory tree, while the second selectively expands states whose observed outcomes suggest greater consequentiality.

Third, we investigate whether targeted counterfactual expansion can improve variable, node-level credit assignment without increasing the total rollout budget.

## 8 Limitations and Future Work

The Phase-I exploration heuristic uses continuation cross-entropy as a proxy for the informativeness of a reasoning state. High continuation cross-entropy indicates surprise in an observed continuation, but does not necessarily imply that alternative continuations will produce different outcomes. Thus, its usefulness as a branching heuristic is an empirical hypothesis that will be evaluated through controlled ablations.

The consequentiality signal

$$\delta(v) = V(v) - V(\text{parent}(v)) \quad (13)$$

is also only one possible measure of decision importance. Future experiments may investigate alternative signals such as the magnitude of value differences, disagreement between child values, or estimates of the value of an additional rollout.

Finally, the initial evaluation focuses on mathematical reasoning because deterministic rewards enable controlled experimentation. Extending the framework to tasks with richer process rewards or interactive environments is left for future work.

## 9 Tentative Timeline

- **September 2026:** Implement segment-wise tree construction and continuation cross-entropy exploration.
- **September–October 2026:** Implement credit-guided counterfactual allocation and variable-credit advantage estimation.
- **October 2026:** Run baseline and ablation experiments.
- **October 2026:** Analyze rollout allocation, credit localization, and reasoning performance.

## References

- Fei Ding, Yongkang Zhang, Yeling Peng, Youwei Wang, Guoxiong Zhou, and Zijian Zeng. 2026. [Reducing credit-assignment variance through counterfactual reasoning paths](#). *arXiv preprint arXiv:2605.16302*.
- Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. 2025. [TreeRL: LLM reinforcement learning with on-policy tree search](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*.
- Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tianshun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, Zheng Zhang, Wei Shen, Qian Liu, Chenghua Lin, Jian Yang, Ge Zhang, and Wenhao Huang. 2025. [TreePO: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling](#). *arXiv preprint arXiv:2508.17445*.
- Hieu Tran, Zonghai Yao, and Hong Yu. 2025. [Exploiting tree structure for credit assignment in reinforcement learning with large language models](#). *arXiv preprint arXiv:2509.18314*.
- Yijun Zhang, Fan Xu, Jiaxin Ding, Yule Xie, Shiqing Gao, Xin Ding, Haoxiang Zhang, Luoyi Fu, and Xinbing Wang. 2026. [Information gain-based rollout policy optimization: An adaptive tree-structured rollout](#)

approach for multi-turn LLM agents. *arXiv preprint arXiv:2607.06223*.

Heming Zou, Qi Wang, Yun Qu, Yuhang Jiang, Lizhou Cai, Yixiu Mao, Ru Peng, Xin Xu, Weijie Liu, Kai Yang, Saiyong Yang, and Xiangyang Ji. 2026. **TRACE: A unified rollout budget allocation framework for efficient agentic reinforcement learning.** *arXiv preprint arXiv:2606.11119*.

---

**Algorithm 1** Credit-Aware Counterfactual Rollout Allocation

---

**Require:** Input problem  $x$ , rollout budget  $n$ , reserved budget  $k$ , temperature  $\tau$ , coefficient  $\lambda$

- 1: Initialize exploratory tree  $T_0 \leftarrow \emptyset$
- 2: Generate a complete trajectory  $\ell_1$  from  $x$
- 3: Add  $\ell_1$  to  $T_0$
- 4: Compute continuation cross-entropies for nodes in  $T_0$
- 5: **for**  $r = 2, \dots, n - k$  **do**
- 6:   Compute  $\mu_v, \sigma_v$  for eligible nodes  $v$
- 7:   Compute  $S_{\text{explore}}(v) = \mu_v + \lambda\sigma_v$
- 8:   Select a branching node according to  $S_{\text{explore}}$
- 9:   Generate a complete continuation from  $v$
- 10:   Add the trajectory to  $T_0$
- 11:   Recompute continuation statistics for affected nodes
- 12: **end for**
- 13: Compute tree values

$$V(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} R_\ell$$

14: Compute consequentiality

$$\delta(v) = V(v) - V(\text{parent}(v))$$

15: Compute allocation distribution

$$p(v_i) = \frac{\exp(\delta(v_i)/\tau)}{\sum_{v_j} \exp(\delta(v_j)/\tau)}$$

- 16: **for**  $j = 1, \dots, k$  **do**
- 17:   Sample  $v \sim p(v)$
- 18:   Set expansion state to  $\text{parent}(v)$
- 19:   Generate an independent continuation from the expansion state
- 20:   Add the trajectory to the existing tree
- 21:   Update the tree
- 22: **end for**
- 23: Recompute tree values and node-level credit signals
- 24: Construct variable-credit advantage estimates
- 25: Perform the policy update
- 26: **return** Final tree  $T$  and updated policy

---